

Spectral Predictor — A Simple Guide

Predictor 3 of 9 · A Unified Benchmark of Grokking Predictors in Neural Networks

- Based on the paper by Canatar, Bordelon and Pehlevan (2021), *Nature Communications* 12, 2914 — “Spectral bias and task-model alignment explain generalization in kernel regression and infinitely wide neural networks.”
- Task studied: $(a + b) \bmod 97$, a decoder transformer trained on PyTorch, on Apple Silicon (MPS)
- Method: 5 different random seeds, each trained for 40,000 epochs, checkpoints studied afterward
- Result: CLOSED — NEGATIVE, and this result is the same in all 5 seeds
- Prepared for: Jonathan John, DevGrid System

The short answer: The Spectral predictor was built correctly. But when we look at the actual training runs, the signal it measures — how much the model's learning is “lined up” with the top directions of its own representation — becomes strongest only **after** the model has already grokked, not before. So this predictor is not a good early-warning sign for grokking on this task. This is not a mistake in the code. It is a real and useful finding: it tells us that grokking on modular addition happens through a different mechanism than the one this theory describes.

Contents

1. What Is Grokking? — A Quick Recap
2. What Is A “Grokking Predictor”, And Why Nine Of Them?
3. Meeting The Spectral Predictor
4. The Big Idea: Spectral Bias And Task-Model Alignment
5. The Mathematics, Step By Step
6. How It Was Coded In This Project
7. How The Experiment Was Set Up
8. The Results, Shown As Graphs
9. The Verdict — Why The Theory Did Not Work Here
10. In The Project's Own Words (from context.md)
11. Points To Remember
12. Check Your Understanding

1. What Is Grokking? — A Quick Recap

Let us first recall what this whole project is about, since the Spectral predictor will make sense only after this is clear.

In this project, a small transformer network is trained to learn modular addition. The task is simple: given two numbers a and b , find $(a + b) \bmod 97$. During training, something curious happens. For a long time, the network memorises the training examples. Its accuracy on new, unseen examples (called test accuracy) stays low, close to random guessing. Then, suddenly, usually much later in training, the test accuracy jumps up sharply and the network starts generalising well. This sudden, delayed jump in generalisation is called **grokking**.

The epoch (that is, the training step) at which this jump happens is called the **grok epoch**. Naturally, researchers want to know: is there some quantity, measurable *before* the grok epoch, that already hints that grokking is about to happen? Such a quantity is called a **grokking predictor**.

New words used in this chapter

Epoch

One complete pass of the training data through the network. Training runs for many epochs, one after another.

Test accuracy

How correct the model's answers are on examples it has never been trained on.

Grokking

A sudden, delayed jump in test accuracy, coming long after the model has already fit its training data well.

Grok epoch

The training epoch at which this sudden jump in test accuracy happens.

2. What Is A “Grokking Predictor”, And Why Nine Of Them?

A grokking predictor is some number, computed from the model, that researchers believe should change in a telltale way *before* the grok epoch. If a predictor is correct, we could, in principle, watch that number during training and say, “Grokking is coming soon,” well before the test accuracy actually jumps.

Over the last few years, different researchers have proposed different predictors, each coming from a different theory of why grokking happens. This project takes nine such predictors from published papers and tests every one of them, fairly and under the same conditions, on the same task. The fixed order in which they are tested is given below. Spectral is the third one in this list.

L2 Norm → Dropout → **Spectral** → AGE → HTSR Alpha → Correlation Traps → Weight-PCA → Higher-MI → Commutator Defect

The first two predictors in this list, L2 Norm and Dropout, are simple numbers read directly from the trained weights. Spectral is more involved. It needs us to look at how the model represents its inputs internally, and to break that representation down using ideas from linear algebra. Let us build up to this slowly.

3. Meeting The Spectral Predictor

The Spectral predictor is based on a paper by Canatar, Bordelon and Pehlevan, published in 2021 in *Nature Communications*. Their paper is not about grokking at all. It is about a simpler and older question in machine learning theory: in a particular, well-studied kind of learning, called kernel regression, when does a model generalise well, and when does it not?

The Spectral predictor takes the answer that Canatar and colleagues gave for kernel regression, and checks whether the same answer also explains grokking in this project's transformer. That is the whole idea in one sentence. The rest of this report explains what "the answer" actually was, how it was computed here, and whether it actually worked.

4. The Big Idea: Spectral Bias And Task-Model Alignment

Let us build up the idea in small steps, using a simple picture before we come to the actual mathematics.

4.1 A network's hidden representation

When the trained transformer is shown a training example, it produces, somewhere inside itself, a list of numbers just before making its final prediction. This list of numbers is called the model's **hidden representation** of that example. If we do this for every training example and collect all these lists together, we get a big table of numbers: one row for every example, one column for every internal number the model computes. Mathematically, this table is a matrix, which this report calls Φ (the Greek letter phi).

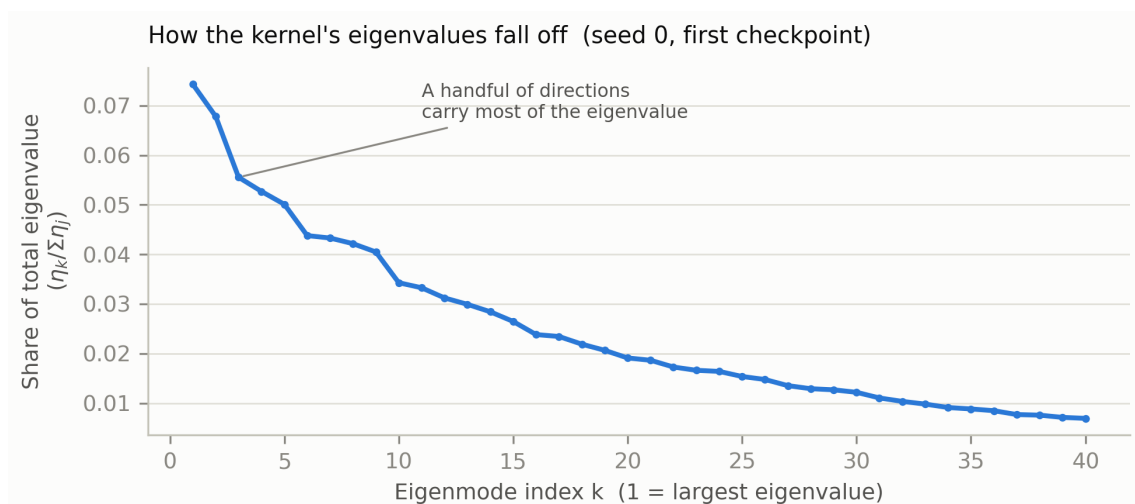
4.2 Finding the “natural directions” of this representation

Now think of every row of this table as a point in space, one point per training example. These points, taken together, form a cloud. This cloud is usually not a neat round ball. It is stretched out more in some directions than others, a little like a chapati that has been rolled out unevenly, more along one direction than another. Linear algebra gives us a way to find these “natural stretching directions” of a cloud of points exactly. These directions are called **eigenvectors**, and the amount of stretching along each direction is called its **eigenvalue**. The direction with the largest eigenvalue is the direction along which the data varies the most.

To find these directions, we first form what is called the model's **kernel**, or **Gram matrix**, written K . This is simply a table that records how similar every pair of training examples is, according to the model's own representation of them. Once we have K , finding its eigenvectors and eigenvalues tells us the natural directions of the representation, and how important each direction is.

4.3 “Spectral bias” — which directions get learned first

Canatar and colleagues' theory says something specific about how a network of this general kind learns: it tends to pick up the large-eigenvalue directions of its kernel first, and the small-eigenvalue directions only later, if at all. This tendency is called **spectral bias**. The chart below shows this idea using real numbers from this project. It plots the eigenvalues of the kernel from seed 0, at the very first checkpoint of training, sorted from the largest to the smallest.



Notice how sharply the curve falls at the start. A small number of directions (on the left) carry a much bigger share of the total eigenvalue than the rest. According to spectral bias, these are the directions the network's representation should be shaped by fastest.

4.4 “Task-model alignment” — does the target function agree?

The second half of the theory asks a different question. Forget the model for a moment, and look only at the labels we want the model to predict — here, the correct value of $(a + b) \bmod 97$ for each training example. This target function also has some structure. We can ask: how much of this target's structure falls along the model's large-eigenvalue directions, and how much falls along its small-eigenvalue directions?

If most of the target's structure lines up with the model's large-eigenvalue directions, the target and the model's kernel are said to be well **aligned**. Canatar's theory predicts that good alignment is exactly what allows fast, sample-efficient generalisation. This is the core claim being tested here: **does task-model alignment rise, and does the number of eigenmodes needed to explain the target fall, before the grok epoch?**

Activity — think about this before moving ahead

Imagine two students revising for an exam. Student A's revision (the “target”) matches closely with the chapters the teacher spent the most time on in class (the “large-eigenvalue directions”). Student B revises chapters the teacher barely touched. Which student's revision is better “aligned” with what was actually taught? Canatar's theory says a student like A should learn faster and score higher. In this report, we check whether the network's grokking follows the same logic.

New words used in this chapter

Hidden representation

The list of internal numbers a trained network computes for a given input, just before producing its final answer.

Kernel / Gram matrix (K)

A table recording, using the model's own representations, how similar every pair of training examples is to each other.

Eigenvector

One of the “natural directions” along which a cloud of data points is stretched.

Eigenvalue

A number telling us how much the data cloud is stretched along a particular eigenvector; a bigger eigenvalue means more stretching, and more importance.

Spectral bias

The tendency of a network to learn the large-eigenvalue directions of its kernel earlier in training than the small-eigenvalue ones.

Task-model alignment

How much of the target function's structure sits along the model's large-eigenvalue directions, rather than its small-eigenvalue ones.

5. The Mathematics, Step By Step

Now that the ideas are clear in words, let us write them down properly, one equation at a time. Each equation below is simply the exact version of an idea already explained above.

5.1 The representation kernel

At a saved checkpoint, the model is run on all N training examples. For each one, its hidden representation is collected, giving the matrix Φ described in Section 4.1. After centring (that is, subtracting the average row so the cloud is centred at zero), the kernel is formed as:

$$K = \Phi_C \Phi_C^\top, \quad \Phi_C = \Phi - \bar{\Phi}$$

5.2 Kernel eigenvalues

Solving the following equation gives the eigenvalues η_k , arranged from largest to smallest, and the matching eigenvectors, collected as columns of U . This is exactly the “natural directions” step from Section 4.2:

$$K u_k = \eta_k u_k, \quad \eta_1 \geq \eta_2 \geq \dots \geq \eta_N \geq 0$$

5.3 How much of the target sits in each direction

The correct labels — here, one-hot vectors for the 97 possible answers — are centred and then projected onto the eigenvector directions found above. The squared length of this projection, for direction k , is called the task power in that direction:

$$W = U^\top Y_C, \quad w_k^2 = \|W_k\|^2$$

Dividing by the total gives the fraction of power carried by direction k :

$$p_k = \frac{w_k^2}{\sum_j w_j^2}$$

5.4 Adding up the power — task-model alignment

Adding up the power of the top k directions gives the cumulative power $C(k)$. This is Canatar's alignment quantity from Section 4.4. If $C(k)$ is already close to 1 for a small k , the target is well aligned with the model's kernel:

$$C(k) = \sum_{j \leq k} p_j$$

5.5 The four numbers actually recorded

For every saved checkpoint, four numbers are computed and stored. k_{90} and k_{95} tell us how many leading directions are enough to explain 90% and 95% of the target's power:

$$k_{90} = \min\{k : C(k) \geq 0.90\}$$

The alignment score is the task power, weighted by how important (large-eigenvalue) each direction is:

$$\text{alignment score} = \sum_k \tilde{\eta}_k p_k, \quad \tilde{\eta}_k = \frac{\eta_k}{\sum_j \eta_j}$$

The entropy tells us how spread out the target's power is across all the directions, instead of being concentrated in a few:

$$H = - \sum_k p_k \log p_k$$

How to read these four numbers together. If Canatar's theory applies here, then as training goes on, k_{90} should fall (fewer directions are enough), the alignment score should rise (power shifts to important directions), and this should all happen *before* the grok epoch, not after it. This is precisely the claim tested in Sections 8 and 9.

6. How It Was Coded In This Project

The predictor is written in the file `src/predictors/spectral.py`. Like the other predictor files in this project, it does not use any shared base class. It is simply wired into `run_nanda_benchmark.py` through a function named `_checkpoint_predictor_spectral()`, and its results are saved through a matching method in `src/unified_measurements.py`.

6.1 The steps performed by the code, in plain words

- 1 **Step 1 — Watch the model from inside.** A small piece of code called a “forward hook” is attached to the model's final layer. Every time the model processes a training example, this hook quietly copies out the hidden representation, without changing the model in any way.
- 2 **Step 2 — Prepare for precise maths.** The copied numbers are moved to the computer's CPU and converted to a more precise number format (float64), because the next step, finding eigenvalues precisely, needs this extra precision and cannot be done directly on the Apple Silicon GPU (called the MPS backend) used for training.
- 3 **Step 3 — Build the kernel.** The representation table Φ is centred, and the kernel $K = \Phi_c \Phi_c^T$ is formed, exactly as in Section 5.1.
- 4 **Step 4 — Find the natural directions.** `torch.linalg.eigh` is called on K to get its eigenvalues and eigenvectors, then these are sorted from largest to smallest.
- 5 **Step 5 — Measure the target's alignment.** The correct labels are centred and projected onto these directions, giving the task power for each one, and then the cumulative power.
- 6 **Step 6 — Save the four numbers.** k_{90} , k_{95} , the alignment score, and the entropy are worked out and saved, along with the first 50 eigenvalues and first 100 cumulative-power values, so that graphs like the ones in this report can be drawn later.

6.2 A small hardware problem that came up, and how it was fixed

While testing this code for the first time, an error appeared: “Cannot convert a float64 Tensor to MPS.” This happened because the program first tried to change the number's precision, and only after that tried to move it off the Apple Silicon GPU — but the GPU does not support that precise a number format at all, so the conversion failed. The fix was simply to do the two steps in the opposite order: move the data off the GPU first, and only then make it more precise.

```
captured["h"].cpu().double() # correct order: move off MPS first, then increase precision
```

6.3 A short history of this file

It is worth knowing that the file used today is not the first attempt. An earlier version of this predictor computed a different, simpler set of numbers directly from the model's weight matrices (things like their singular values and “stable rank”). That version was inspired by a different paper altogether, about a low-rank circuit found by Nanda and colleagues, and it was not really an implementation of Canatar's theory. Once this mix-up was noticed, the old code was removed completely, its earlier (negative) result was set aside as not meaningful, and the predictor was rebuilt faithfully to match Canatar's theory, as described in this report.

Remember

- The Spectral predictor watches the model's own hidden representation, not just its weights.
- It needs an eigendecomposition of a kernel, which is a heavier calculation than the earlier predictors in this project (L2 Norm, Dropout) needed.
- The eigendecomposition step must run on the CPU in float64, because Apple's MPS backend cannot do it directly.
- The current file is a faithful, corrected rebuild of an earlier, mistaken version.

7. How The Experiment Was Set Up

To test the predictor fairly, it was measured on real training runs, not just on a quick example. The table below lists exactly how the experiment was run.

Task	(a + b) mod 97, modular addition
Architecture	Decoder transformer (the same transformer.py file shared by every predictor)
Training length	40,000 epochs, for each seed
Number of seeds	5 (numbered seed 0 to seed 4), each trained separately from a different starting point
Training-example count (N)	3,830 examples (about 0.3 × 113 × 113)
Checkpoints studied	24 per seed, spread out across the whole training run
Hardware	Apple Silicon, using PyTorch's MPS backend (with the eigendecomposition done on the CPU)

Why five different seeds, and not just one? Training a neural network involves some randomness — for example, in how its weights are first set up. If we tested the predictor on only one training run, we could not be sure whether our finding was a general pattern or just a one-off coincidence. Testing on five independent runs lets us check whether the same pattern shows up again and again.

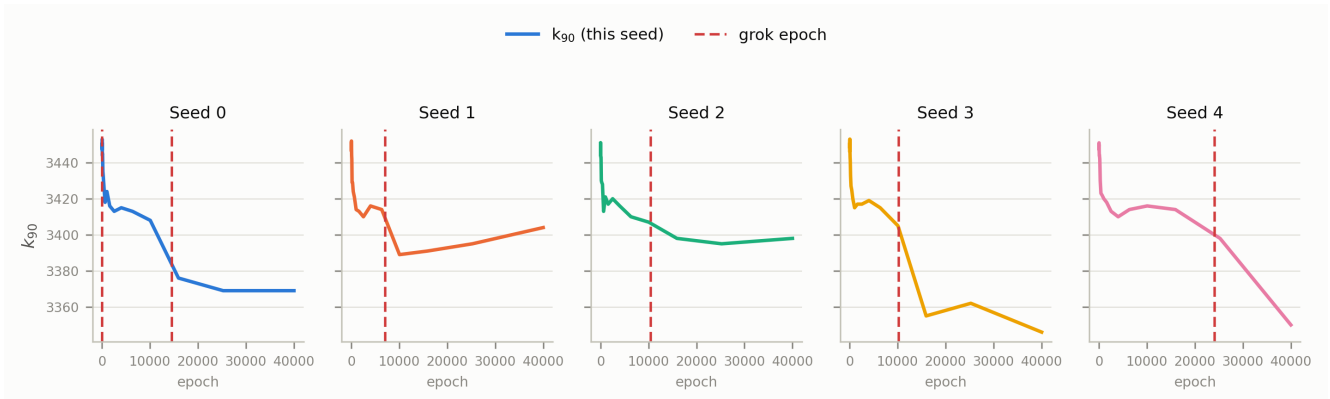
The exact question being asked: does k_{90} reach its lowest value, and does the alignment score reach its highest value, at an epoch that comes *before* that seed's grok epoch?

8. The Results, Shown As Graphs

The grok epoch for each of the 5 seeds turned out to be: 14474, 6988, 10418, 10193, and 24021. On average this is about 13,219 epochs, but notice how much this varies from one seed to another — this by itself tells us that grokking's exact timing is somewhat unpredictable, which is exactly why a good predictor would be so useful.

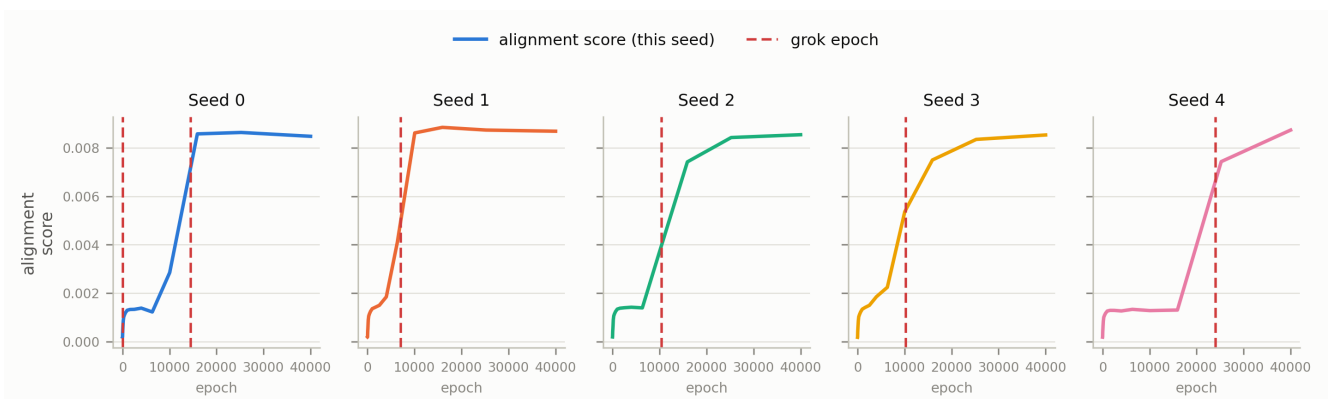
8.1 How k_{90} changed during training

Wait — before reading on, recall that k_{90} is the number of leading directions needed to explain 90% of the target's power. If Canatar's theory works here, this number should fall clearly, and it should fall *before* the red dashed line (the grok epoch) in each of the five small graphs below.



Looking at these five graphs together, k_{90} does drift downward a little as training proceeds, but the drop is small, and in every seed, most of the drop happens close to or after the red line, not clearly before it. In seed 0, for example, k_{90} only reaches its lowest recorded value around epoch 25,232, well after the grok epoch of 14,474.

8.2 How the alignment score changed during training



This graph is more encouraging at first glance — the alignment score does rise sharply around the same time as grokking, in every seed. However, looking carefully, in every single seed, the score keeps climbing a little further even after the red line, and its highest recorded value is reached only after the grok epoch, not before it. So while alignment and grokking happen around the same time, alignment does not clearly come first, the way the theory would need it to.

8.3 The numbers, side by side

Seed	k90 first → last	Epoch of min k90	Alignment first → last	Epoch of max alignment	Grok epoch	k90 leads grok?
0	3448 → 3369	25232	0.0002 → 0.0085	25232	14474	No
1	3447 → 3404	10040	0.0002 → 0.0087	15917	6988	No
2	3448 → 3398	25232	0.0002 → 0.0085	39999	10418	No
3	3450 → 3346	39999	0.0002 → 0.0085	39999	10193	No
4	3448 → 3350	39999	0.0002 → 0.0087	39999	24021	No

In every one of the 5 seeds, the epoch of minimum k_{90} is at or after the grok epoch, and the epoch of maximum alignment score is after the grok epoch.

For seed 0, the ratio between the epoch of minimum k_{90} (25,232) and the grok epoch (14,474) is about 1.74. In simple words, the predictor's own signal takes about 74% longer than the grokking jump itself to reach its extreme value — it clearly follows grokking, rather than warning us about it in advance.

The saved result files for each seed live under `results/nanda_unified/seed_<i>/spectral/`, and include `spectral_signal.json` together with several `.npy` files (the eigenvalues, k_{90} , k_{95} , entropy, alignment score, and checkpoint numbers), which is exactly the data used to draw the graphs above.

9. The Verdict — Why The Theory Did Not Work Here

CLOSED — NEGATIVE (the same standing as L2 Norm and Dropout)

Let us be very clear about what this negative result means, and what it does not mean. It does **not** mean there is a bug in the code. The small, late drop in k_{90} , and the late peak in the alignment score, are exactly what a correctly working Canatar-style calculation should show on a task like this one. The result is negative in the sense that the predictor fails to do the one job it was meant to do: warn us about grokking in advance.

9.1 Two different ways of learning

To understand why the theory does not apply here, it helps to know that there are, broadly, two different ways a network can learn.

In the first way, called the **lazy, or kernel, regime**, the network's internal representation barely changes during training. Only a simple readout layer on top of this fixed representation is adjusted. Canatar's theory was written for exactly this situation — a fixed kernel, and a target being fitted on top of it.

In the second way, called the **feature-learning regime**, the network's internal representation itself keeps changing and improving as training goes on. This project's earlier work (following Nanda and colleagues) has already shown that grokking on modular addition happens because the network is slowly building a very particular internal structure, related to Fourier analysis of the numbers involved, deep inside its representation. This is feature learning, not lazy learning.

Because Canatar's theory assumes a fixed kernel, and this project's transformer keeps rebuilding its kernel right up to (and after) the grok epoch, the theory's central assumption simply does not hold here. That is the real, underlying reason the predictor fails: not because of an error anywhere in the code, but because the situation it was designed for is a different one from what is actually happening inside this network.

Activity — an everyday comparison

Think of a tailor who has already decided on a fixed pattern, and is only adjusting the final stitching to fit a customer better — this is like the lazy regime, where the model's representation is fixed. Now think of a tailor who, while stitching, keeps redesigning the pattern itself based on what fits well — this is like feature learning, where the representation itself changes. Canatar's theory describes how the first tailor works. The transformer in this project behaves more like the second tailor. This is why a theory that works well for the first tailor need not work for the second one.

Remember

- A negative result on a well-implemented predictor is a genuine scientific finding, not a failure of the project.
- Canatar's theory (lazy/kernel regime) does not match how this transformer actually learns (feature-learning regime).
- The representation kernel here stays close to full rank right through the grok transition — it does not collapse onto a few directions ahead of time, as the theory would need.
- This finding fits together with the negative results already found for L2 Norm and Dropout, building a consistent picture across the first three predictors tested.

10. In The Project's Own Words (from context.md)

For an accurate historical record, here are some lines taken directly, word for word, from the project's running notes file, `context.md`.

"The previous Spectral predictor (Predictor 3 of 9) was a per-weight-matrix SVD proxy — `svdvals(W)` on each weight matrix, then `spectral_norm`, `fro_norm`, `stable_rank`, `effective_rank`. That was inspired by Nanda's low-rank Fourier circuit, NOT by Canatar et al. Its CLOSED-negative verdict from the Sept 5 session ... was therefore a verdict on the wrong metric."

"Spectral (Predictor 3 of 9): re-implemented, NOT yet evaluated. The old CLOSED-negative verdict does NOT carry over — it was about the weight-SVD proxy, which no longer exists. Spectral is back to 'built, awaiting its 5-seed run'."

"Smoke numbers (100-epoch, ungrokked model, so plumbing only — NOT a scientific result): $k_{90} \approx 3449$ ($\approx 0.9 \cdot N$), entropy ≈ 8.24 ($\approx \log N$) — task power is near-uniform across eigenmodes for an untrained net, as expected."

"Criterion 1 — does the alignment signal lead grok? No. `k_90_min_epoch` is at or after `grok_epoch` in all 5 seeds (seed 1 is 10040 vs grok 6988 — after; the rest are well after, three pinned at the last checkpoint 39999). `alignment_max_epoch` is after grok in 5/5."

"The scientific reason the predictor fails here: Spectral (Canatar) fails because Canatar et al.'s theory is a kernel / NTK (lazy regime) result ... Grokking on modular addition is a feature-learning regime phenomenon ... This is a valid falsification of the predictor on this benchmark, not an implementation failure."

11. Points To Remember

- Grokking is a sudden, delayed jump in test accuracy, coming well after training accuracy is already high. The epoch when this jump occurs is the grok epoch.
- A grokking predictor is a number that should change in a telltale way before the grok epoch, warning us in advance.
- The Spectral predictor is based on Canatar et al. (2021), and measures spectral bias and task-model alignment using the model's own hidden representation, not just its raw weights.
- It works by building a kernel from the representation, finding its eigenvalues and eigenvectors, and measuring how much of the target's structure lines up with the important (large-eigenvalue) directions.
- It was tested honestly, across 5 independent seeds, each trained for the full 40,000 epochs.
- In all 5 seeds, the predictor's key numbers (k90 and the alignment score) reach their most extreme values at or after the grok epoch, never clearly before it.
- The verdict is CLOSED, NEGATIVE — this is a genuine scientific finding, not a coding mistake.
- The likely reason: Canatar's theory assumes a fixed representation (the lazy regime), while grokking on modular addition happens through active feature learning, where the representation itself keeps changing.
- The next predictor in the fixed order is AGE (Predictor 4), which has also been closed with a negative verdict in this project. HTSR Alpha (Predictor 5) is marked as the next one to begin.

12. Check Your Understanding

Before moving on to the next predictor, try answering these questions in your own words. They are meant to check whether the main ideas of this report are clear, not to test memorisation.

1. In your own words, what is the difference between test accuracy and training accuracy, and why does grokking involve a gap between the two?
2. What do the eigenvectors of the representation kernel represent, in plain language?
3. If task-model alignment were a good predictor of grokking here, what pattern would you expect to see in the alignment-score graphs of Section 8.2? Did the actual graphs show this pattern?
4. Why can the eigendecomposition step not be run directly on the Apple Silicon MPS backend, and what was done instead?
5. In your own words, explain the difference between the lazy (kernel) regime and the feature-learning regime, and say which one better describes this project's transformer.

Source: `src/predictors/spectral.py`, `context.md`, and `results/nanda_unified/` in the grokking-benchmark project.